

Penetration Testing Report

Mediroza General Hospital

NetworkWalks Internship	Batch B083 Week 4
Target	https://medirozahospital.com
Engagement Type	Black-box Penetration Test
Duration	5 Days
Tester	Noman Atiq (Gh0st)

Personal data uncovered during testing (patient identifying details and staff national ID numbers) has been redacted from the evidence in this report to protect the individuals involved, consistent with how such data would be handled in a real engagement. Underlying findings and their severity are unaffected by the redaction.

1. Executive Summary

When this engagement was assigned, I saw it as more than just a weekly task — it was also a chance to test out the vulnerability scanner I built for my own PGD Cyber Security project against a live, authorized target. The scanner flagged 35 findings, including what looked like a serious one: Spring4Shell (CVE-2022-22965), rated High with a CVSS of 8.0.

That result turned out to be the most interesting lesson of the week. I wasn't able to get it to execute through Metasploit, and when I looked closer at the actual tech stack (PHP on LiteSpeed, no Java or Spring anywhere in sight), it became clear this was a false positive — the scanner had pattern-matched on a URL string without understanding the underlying application. I verified this manually and documented it as such rather than reporting it as a real finding.

What the scan did get right, even indirectly, was pointing me toward robots.txt, which disallowed a few interesting paths. Chasing those down led to the actual breach: a SQL injection authentication bypass on the patient portal, which exposed three encrypted pathology PDF reports, and a separate exposed database backup file sitting in an /old/ directory that contained the hospital's full staff salary records and shareholder ownership data. Both were far more damaging than the scanner's one loud-but-hollow finding.

Overall, I gained access to confidential patient lab reports, internal HR/financial records, and personally identifiable staff data (including national ID numbers), all without any valid credentials. The hospital's patient-facing authentication is critically weak, and sensitive internal backups are publicly reachable with no access control at all.

2. Scope and Methodology

Target	https://medirozahospital.com (resolved to 199.188.201.16)
Authorization	Written permission granted by the client for this engagement
Rules of Engagement	Domain-limited; no social engineering; no DoS; no out-of-scope testing

Methodology

My approach followed a standard black-box methodology: passive reconnaissance, active enumeration, vulnerability validation, exploitation, and post-exploitation data review.

Reconnaissance

whois lookup identified the domain as registered via NameCheap with privacy protection, created August 2026. nslookup resolved the target to 199.188.201.16. whatweb identified a LiteSpeed server with standard HTML5 content. An automated scan using a custom vulnerability scanner built for my own PGD project returned 35 findings across header misconfigurations, service exposure, and one Spring4Shell flag.

Active Scanning

nmap was run against the full port list matching the automated scan's findings (21, 22, 23, 25, 80, 443, 3306, 3389, 5432, 5900, 6379, 8080, 8443, 8888, 9200, 27017). Results were inconsistent across repeated scans: ports 21 and 23 flipped between open, filtered, and refused depending on timing. Port 80 is fronted by HAProxy and 443 by an openresty/LiteSpeed stack, with a TLS certificate issued to the shared-hosting wildcard domain

(*web-hosting.com) rather than Mediroza specifically, confirming shared infrastructure. wafw00f confirmed a LiteSpeed WAF in front of the application. gobuster directory enumeration identified /staff/, /old/, and wp-admin as noteworthy paths among a larger set of generic shared-hosting artifacts. A later re-run for evidence-capture purposes was blocked by an adaptive anti-automation mechanism returning randomized-length HTTP 200 responses for nonexistent paths — behavior not present during the original successful scan, suggesting the target's protections escalated in response to sustained scanning activity during the engagement.

Vulnerability Validation

The Spring4Shell flag was manually verified via curl against the exact flagged URL and query string; the server returned an unmodified static robots.txt response, confirming the parameter was not being processed by any Java/Spring component. sqlmap was run against the patient portal login form and did not flag either field as injectable, logging multiple HTTP 403 responses consistent with WAF interference. A manual authentication-bypass-style SQL injection attempt against the same form succeeded despite the automated tool's negative result, demonstrating that automated scanning alone is not sufficient to rule out this class of vulnerability.

Exploitation and Post-Exploitation

Unauthorized access to the patient portal was gained via the manual SQL injection bypass. Three password-protected pathology PDF reports were downloaded and each cracked individually, since no single wordlist or tool worked for all three. A separate exposed, unauthenticated database backup file was discovered containing hospital staff and shareholder records. The same SQL injection technique and a custom credential wordlist were attempted against the separate staff login portal; this was not successfully breached within the engagement window.

3. Findings and Proof of Exploitation

Finding 1 — SQL Injection Authentication Bypass (Patient Portal)

Location: <https://medirozahospital.com/patient/login.php>

The patient portal login form was vulnerable to a classic SQL injection authentication bypass. Submitting **admin'--** in the username field with an empty password field bypassed authentication entirely and granted access to the "My lab reports" page, which should only be reachable by an authenticated patient.

An automated sqlmap scan against the same form, using both automatic form-detection and manually-supplied POST data, did not detect this as an injectable parameter, returning "does not seem to be injectable" for both fields, with multiple HTTP 403 responses logged along the way. The manual bypass succeeded regardless, underscoring that automated tooling can miss simple logic flaws that a manual tester will catch.

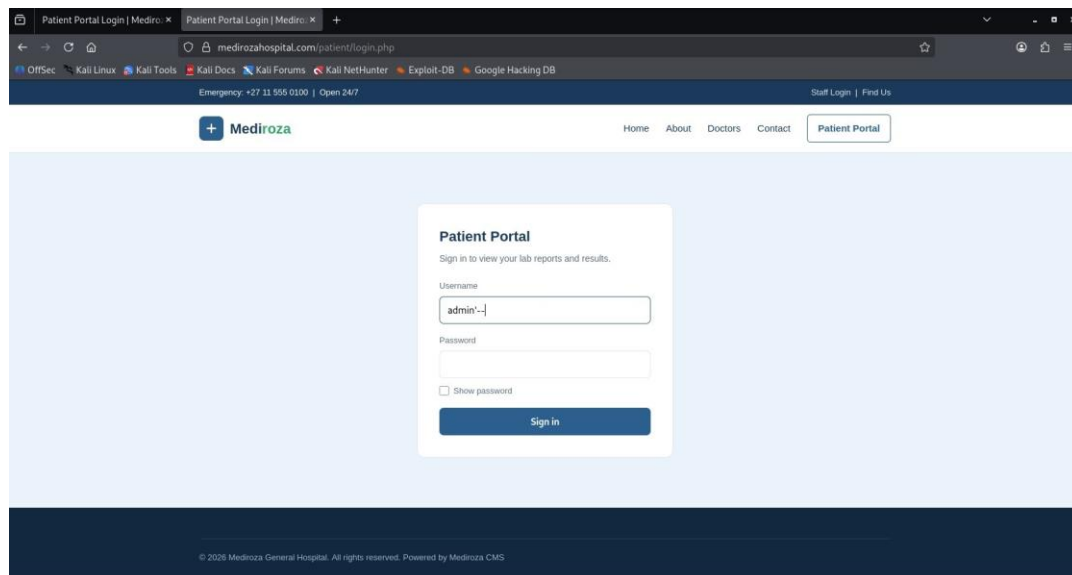


Figure 1 — SQL injection payload (admin'--') entered in the patient login username field

Impact: Any unauthenticated user can access any patient's lab report listing without valid credentials — a direct breach of patient confidentiality and likely relevant healthcare data protection regulation, depending on jurisdiction.

Finding 2 — Exposure of Encrypted Patient Pathology Reports

Location: <https://medirozahospital.com/patient/portal.php>

Once inside the portal (via Finding 1), three pathology lab reports were listed and downloadable without further authentication.

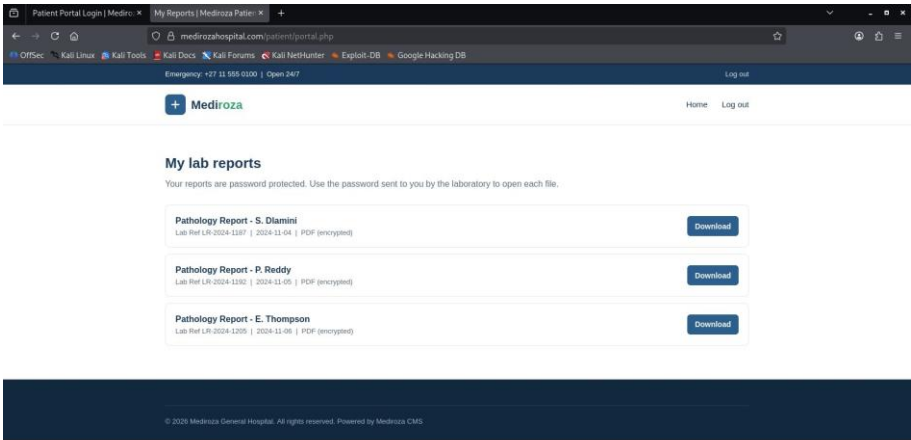


Figure 2 — Patient portal listing the three downloadable, password-protected lab reports

File	Patient	Password	Method
patient_report_1.pdf	S. Dlamini	123456	Dictionary attack (NetworkWalks cracker)
patient_report_2.pdf	P. Reddy	password	Dictionary attack (NetworkWalks cracker)
patient_report_3.pdf	E. Thompson	!@#\$\$%^&	Custom symbol-pattern wordlist (rockyou/John found nothing)

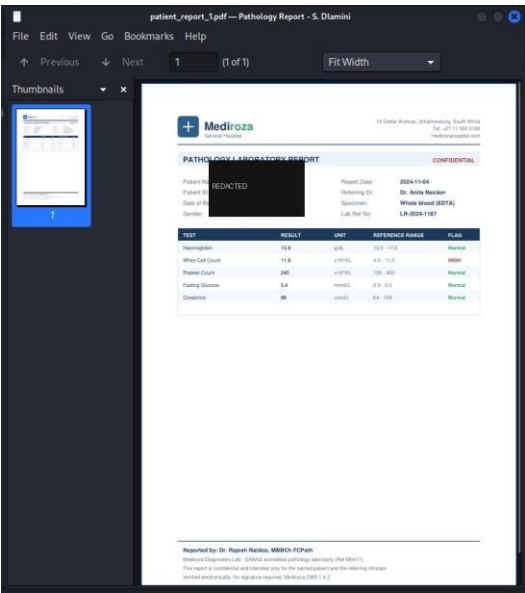


Figure 3a — Decrypted pathology report (patient details redacted)

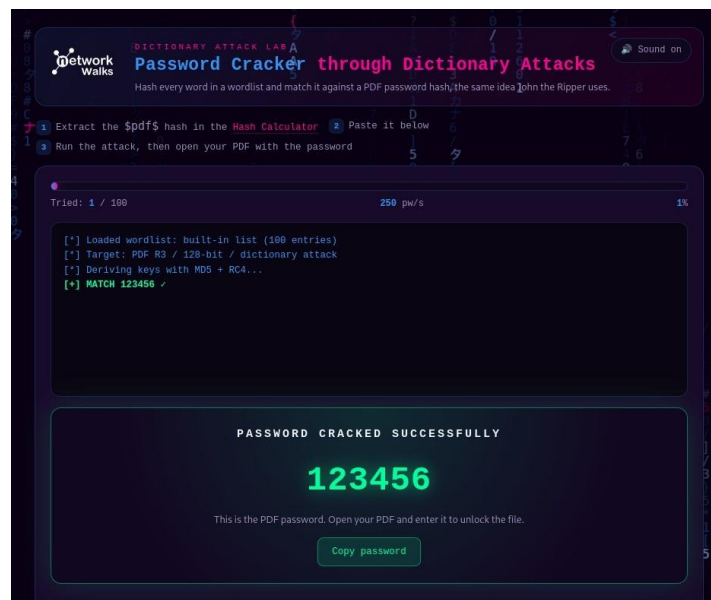


Figure 3b — Successful password crack for patient_report_1.pdf

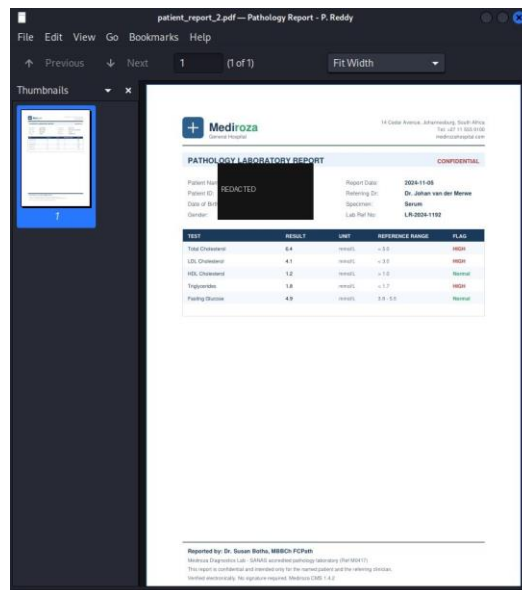


Figure 4a — Decrypted pathology report (patient details redacted)

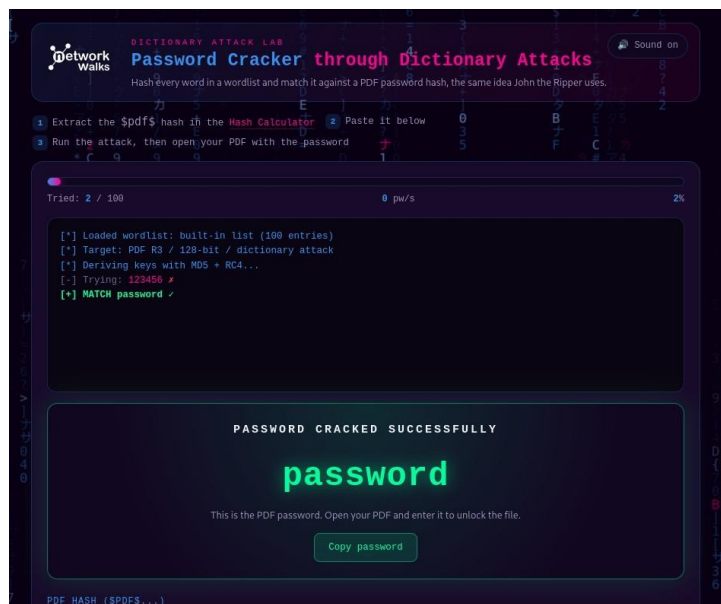


Figure 4b — Successful password crack for patient_report_2.pdf

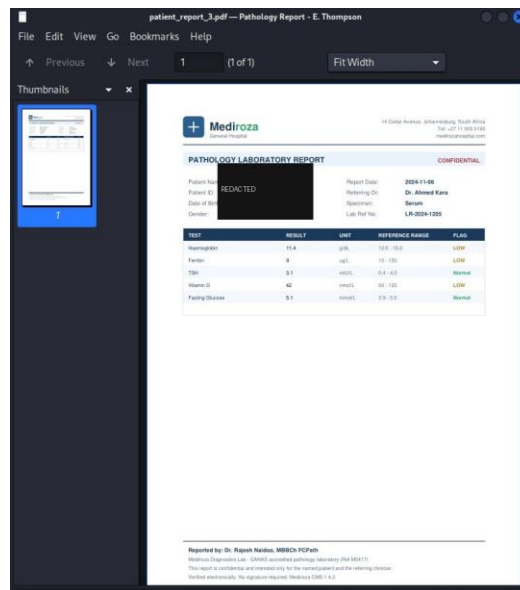


Figure 5a — Decrypted pathology report (patient details redacted)

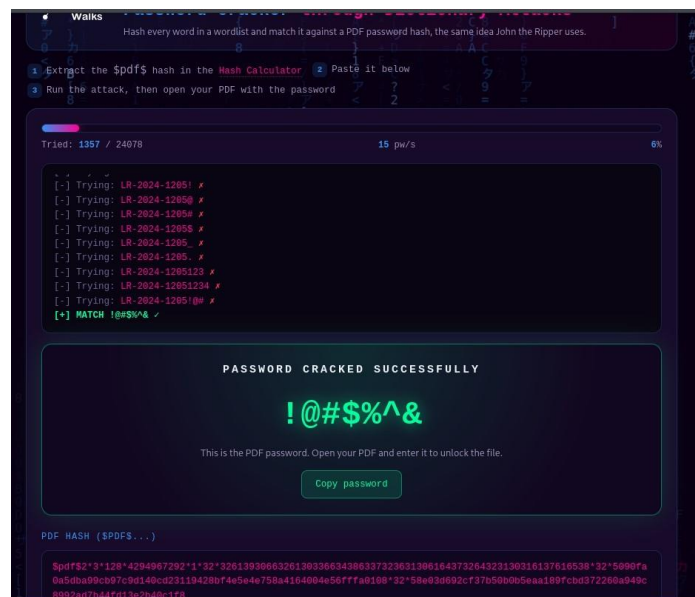


Figure 5b — Successful password crack for patient_report_3.pdf

Each report contained full patient identifying information: full name, date of birth, patient ID, referring physician, and detailed lab results.

Impact: Full confidential patient health records exposed, including identifiable personal and medical data. Two of the three passwords were trivially weak; the third, while non-dictionary, was still crackable with modest effort.

Finding 3 — Exposed, Unauthenticated Database Backup File

Location: https://medirozahospital.com/old/mediroza_db_backup_2019.sql

Directory enumeration identified an /old/ path on the server. Direct navigation to this path revealed a full SQL database backup file, mediroza_db_backup_2019.sql, generated by "Mediroza CMS 1.4.2" and dated 2019-08-27. The file was served in plaintext with no authentication, access control, or encryption of any kind.

The backup contained two full tables: **staff** — 30 records including full name, job title, department, email, phone number, national ID number, monthly salary (ZAR), and date joined, for every employee from the Chief Pathologist and Medical Director down to ward clerks and receptionists; and **shareholders** — 10 records detailing shareholder names, ownership percentage, shares held, and share class, including hospital executives and an external holding company and family trust.

Name	Job Title	Department	National ID	Monthly Salary
Dr. Rajesh Naidoo	Chief Pathologist	Diagnostics Lab	[REDACTED]	[REDACTED]
Dr. Johan van der Merwe	Medical Director	Management	[REDACTED]	[REDACTED]
Sarah Botha	Chief Financial Officer	Finance	[REDACTED]	[REDACTED]
Jameel Malik	IT Systems Administrator	IT	[REDACTED]	[REDACTED]
... (30 staff records total)				

Figure 6a — Sample of exposed staff records (national ID and salary redacted; full table in repo evidence)

Shareholder	Share %	Shares Held	Class
Dr. Rajesh Naidoo	18.0	180,000	Ordinary
Cedar Health Holdings (Pty) Ltd	15.0	150,000	Ordinary
Dr. Johan van der Merwe	12.0	120,000	Ordinary
Reddy Family Trust	11.0	110,000	Ordinary
... (10 shareholder records total)			

Figure 6b — Sample of exposed shareholder records (full table in repo evidence)

Impact: This is the most severe finding of the engagement. Beyond the obvious confidentiality breach of salary and ownership data, the inclusion of full national ID numbers for every staff member creates a direct identity-theft risk independent of anything else disclosed. The file's age (2019) combined with its public reachability suggests it is a forgotten artifact from a site migration or cleanup that was never properly removed or access-restricted.

Finding 4 — Spring4Shell (CVE-2022-22965) — False Positive

Location: Flagged at /robots.txt?class.module.classLoader.DefaultAssertionStatus=...

The initial automated scan flagged this as a High-severity (CVSS 8.0) finding. Manual verification found no evidence to support it: the target's confirmed technology stack is PHP running on LiteSpeed, not Java/Spring/Tomcat. A Metasploit exploitation attempt against the flagged endpoint failed at the configuration stage, unable to even identify the correct HTTP method. A direct curl request to the exact flagged URL and query string returned a standard, unmodified robots.txt response, confirming the query parameter had no effect and was not being parsed by any class-loader logic.

```
msf exploit(multi/http/spring_framework_rce_spring4shell) > show options
Module options (exploit/multi/http/spring_framework_rce_spring4shell):


| Name         | Current Setting                | Required | Description                                                                                                           |
|--------------|--------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| HTTP_METHOD  | Automatic                      | no       | HTTP method to use (Accepted: Automatic, GET, POST)                                                                   |
| PAYLOAD_PATH | webapps/ROOT                   | yes      | Path to write the payload                                                                                             |
| Proxies      |                                | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks5, socks5h, sapni, http, socks4 |
| RHOSTS       | 199.188.201.16                 | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                |
| RPORT        | 8080                           | yes      | The target port (TCP)                                                                                                 |
| SSL          | false                          | no       | Negotiate SSL/TLS for outgoing connections                                                                            |
| TARGETURI    | /app/example/HelloWorld.action | yes      | The path to the application action                                                                                    |
| VHOST        |                                | no       | HTTP server virtual host                                                                                              |


Payload options (generic/shell_reverse_tcp):


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.0.0.2        | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |


Exploit target:


| Id | Name |
|----|------|
| 0  | Java |


View the full module info with the info, or info -d command.
msf exploit(multi/http/spring_framework_rce_spring4shell) >
```

Figure 7 — Metasploit exploit attempt against the Spring4Shell module, aborted due to failure

Conclusion: This finding is a confirmed false positive from the automated scanner, which appears to have pattern-matched on a known exploit URL string without validating the underlying application stack. It is retained in this report to demonstrate verification methodology and to caution against acting on unverified automated scan output.

Finding 5 — Inconsistent/Adaptive Network-Layer Filtering

Location: Ports 21 (FTP) and 23 (Telnet); site-wide directory enumeration

Repeated nmap scans against the same ports returned different results within minutes of each other — port 21 was reported "open" with a Pure-FTPd banner on one scan, then refused a direct client connection seconds later, then reported "filtered" on an immediate re-scan. A later gobuster directory scan, re-run purely to capture clean evidence, was blocked entirely by a wildcard anti-automation response that had not been present during the original successful scan earlier in the engagement.

Impact: While not independently exploitable, this indicates adaptive rate-limiting or intrusion-prevention behavior that escalated in response to sustained scanning activity. Relevant to the client's defensive posture, but also means automated scan results against this host cannot be treated as point-in-time-reliable without manual confirmation.

Finding 6 — Staff Portal (Untested Positive)

Location: <https://medirozahospital.com/staff/login.php>

A separate staff-only login portal was identified via directory enumeration. The same SQL injection technique that succeeded against the patient portal was attempted here and failed; a custom credential wordlist built from known staff data was also attempted without success. The portal returns a generic "Invalid username or password" message that does not distinguish between an invalid username and an invalid password, which is good practice and likely contributed to resisting the testing performed.

Impact: None confirmed. Documented for completeness and to note this as an area warranting further testing beyond the engagement window.

4. Risk Rating

#	Finding	Risk	Justification
1	SQL Injection Auth Bypass (Patient Portal)	Critical	Unauthenticated attacker can access any patient's data with no credentials;
2	Exposure of Patient Pathology Reports	Critical	Direct breach of patient health information; two of three passwords trivially guessed
3	Exposed DB Backup (Staff + Shareholder Data)	Critical	Full PII including national ID numbers, salary, and ownership data, publicly available
4	Spring4Shell (False Positive)	Informational	No confirmed impact; documented for scanner-accuracy awareness
5	Inconsistent Filtering / Adaptive WAF	Low	Not independently exploitable; relevant to scan reliability and defensive posture
6	Staff Portal (Untested Positive)	Informational	No vulnerability confirmed; flagged for further testing

5. Recommendations and Remediation

Finding 1 (SQL Injection)

- Replace all raw/concatenated SQL query construction with parameterized queries or prepared statements across every login form on the site.
- Apply the same fix to the staff login portal proactively, even though it resisted testing, since it may share the same underlying codebase.
- Conduct a full source-code review of all database-interacting endpoints, not just the ones identified in this engagement.

Finding 2 (Weak Patient PDF Passwords)

- Enforce a minimum password complexity policy for any auto-generated document passwords.
- Consider replacing password-protected PDFs with an authenticated download mechanism instead of relying on document-level encryption alone.

Finding 3 (Exposed Backup File)

- Immediately remove the /old/ directory and the backup file from public web-root access.
- Audit the entire web root for other legacy/forgotten files using the same directory enumeration method used in this engagement.
- Store database backups outside the web-accessible directory structure, restricted to authorized administrative personnel only.
- Given the exposure of national ID numbers, assess obligations under applicable data protection and breach notification regulations.

Finding 4 (Scanner False Positive)

- No remediation needed on the application side. Recommend cross-verifying automated scan findings manually before escalating or budgeting remediation effort.

Finding 5 (Adaptive Filtering)

- Continue current WAF/rate-limiting posture; consider extending similar protections consistently across all exposed services, or disabling unused services entirely.

Finding 6 (Staff Portal)

- Recommend a follow-up, credentialed test of the staff portal given its sensitivity, ideally with client-provided test credentials to assess post-authentication controls.